

Báo cáo kết quả huấn luyện

Mô hình dịch Việt - Lào

THỰC HIỆN BỞI

Lương Mạnh Linh	22021163
Đặng Thanh Quang	22021134
Lê Quang Thắng	22021209

Các nội dung chính

- Vấn đề đặt ra
- Mô hình sử dụng
- Quá trình huấn luyện
- Đánh giá chung về mô hình



Vấn đề đặt ra

Tạo mô hình dịch ngôn ngữ Việt-Lào từ đầu

Bộ dữ liệu được cung cấp VLSP2023

- Bộ train: gồm 100.000 cặp câu
- Bộ dev: gồm 2.000 cặp câu
- Bộ test: gồm 1.000 cặp câu

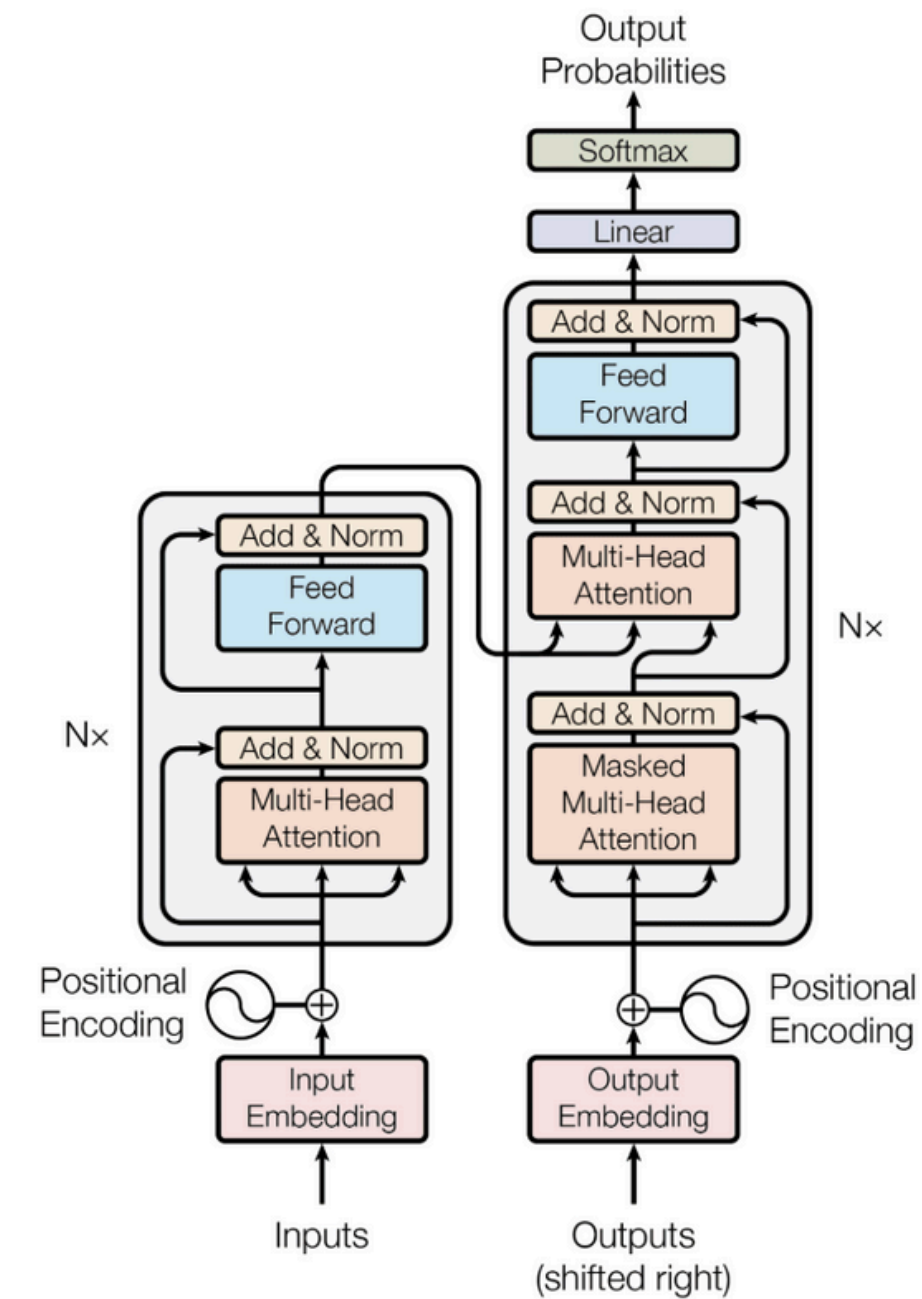


Kiến trúc sử dụng

Kiến trúc sử dụng được hướng đến là transformer

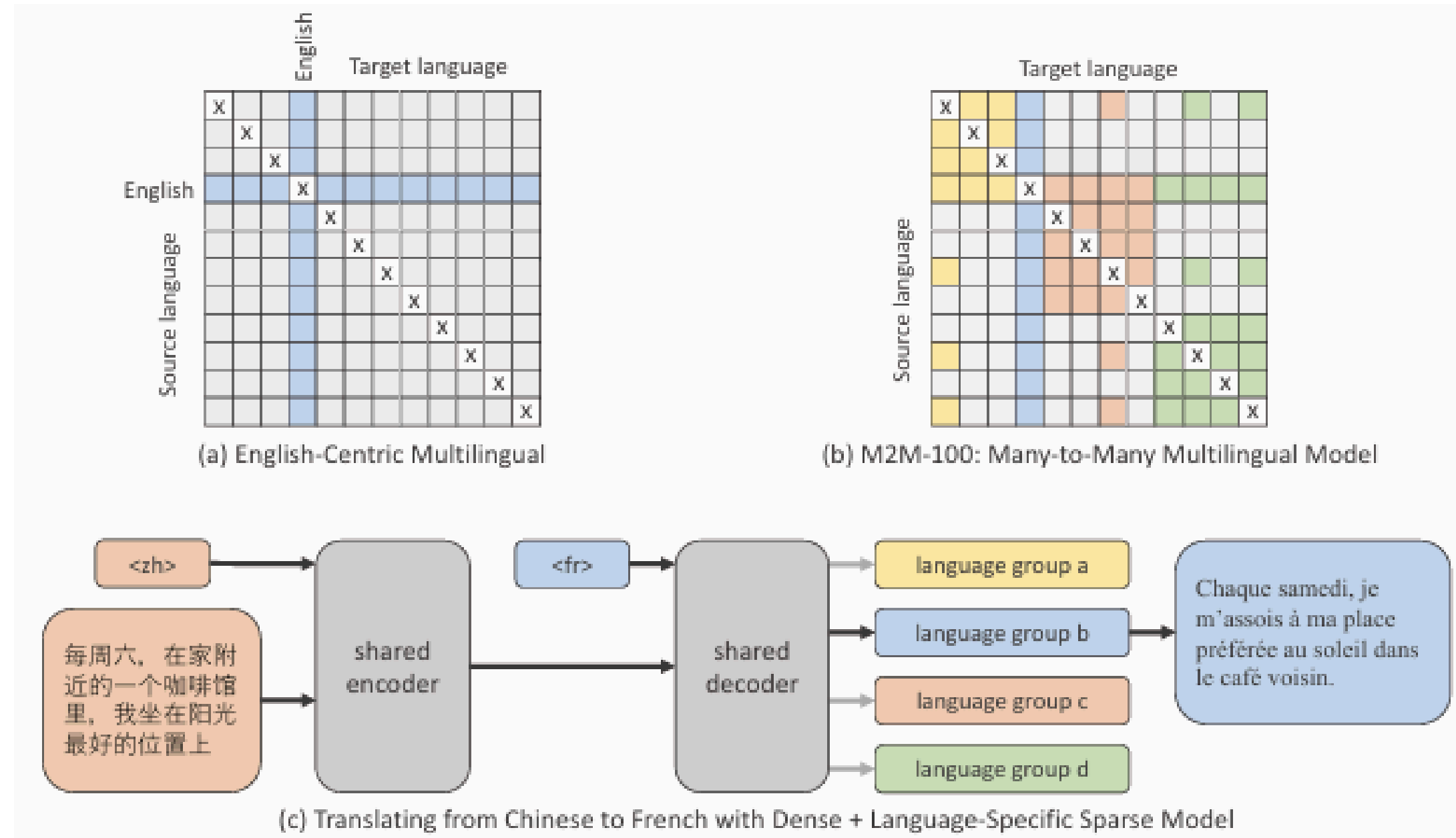
Khi cài đặt và sử dụng kiến trúc transformer chuẩn từ đầu được kết quả tương đối tốt nhưng vẫn chưa đạt mong muốn

→ Sử dụng mô hình với kiến trúc transformer tốt để đạt hiệu quả tốt hơn



Kiến trúc transformer

Mô hình đề xuất



Đề xuất mô hình M2M100 sử dụng kiến trúc transformer

Quá trình huấn luyện

Bộ dữ liệu chuẩn bị

- Bộ dữ liệu VLSP: 100.000 cặp câu.
- Bộ dữ liệu sử dụng Google Translate: 100,000 cặp câu
- Bộ dữ liệu sinh từ Gemini: gần 75,000 cặp câu

Vietnamese_Lao (3 directories)

About this directory

This file does not have a description yet.



GeminiTranslate
2 directories



GoogleTranslate
1 directories



VLSP2023
3 directories

Quá trình huấn luyện

Tổng quan quá trình huấn luyện

- Tải dữ liệu train
- Train sentencepiece model
- Tạo tokenizer và model
- Train model
- Evaluate model

Quá trình huấn luyện

Train sentencepiece model: Sử dụng dữ liệu trong tập Train của VLSP2023 để xây dựng vocab và sentencepiece model.

```
spm.SentencePieceTrainer.train(  
    input='combined_text.txt',  
    model_prefix=model_prefix,  
    vocab_size=vocab_size,  
    character_coverage=0.9995,  
    model_type='unigram',  
    normalization_rule_name='nmt_nfkc',  
    add_dummy_prefix=False,  
    pad_id=0,  
    unk_id=1,  
    bos_id=2,  
    eos_id=3  
)
```

Quá trình huấn luyện

Tokenizer: Tokenizer được tạo từ vocab và sentencepiece model, Model được tạo từ Config và không sử dụng pretrained

```
class Tokenizer(M2M100Tokenizer):
    def __init__(self, vocab_file, spm_file, src_lang='vi', tgt_lang='lo', **kwargs):
        super().__init__(vocab_file=vocab_file, spm_file=spm_file, **kwargs)

        lang_tokens = [value for _, value in self.id_to_lang_token.items()]

        self.add_special_tokens({
            'pad_token': '<pad>',
            'unk_token': '<unk>',
            'eos_token': '</s>',
            'bos_token': '<s>',
            'additional_special_tokens': lang_tokens
        })

        self.src_lang = src_lang
        self.tgt_lang = tgt_lang
```

```
self.config = M2M100Config(
    vocab_size=len(tokenizer),
    d_model=768,
    encoder_layers=6,
    decoder_layers=6,
    encoder_attention_heads=8,
    decoder_attention_heads=8,
    encoder_ffn_dim=2048,
    decoder_ffn_dim=2048,
    max_position_embeddings=512,
    pad_token_id=tokenizer.pad_token_id,
    bos_token_id=tokenizer.bos_token_id,
    eos_token_id=tokenizer.eos_token_id,
    activation_function="gelu",
    scale_embedding=True,
    attention_dropout=0.1,
    activation_dropout=0.1,
    dropout=0.2
)
```

Quá trình huấn luyện

Train model: Sử dụng Seq2SeqTrainer của HuggingFace, thiết lập các thông số trong quá trình train sao cho thích hợp với thiết bị sử dụng và kích thước của model.

```
training_args = Seq2SeqTrainingArguments(  
    report_to="tensorboard",  
    output_dir=output_dir,  
    eval_strategy="epoch",  
    # eval_steps=1000,  
    learning_rate=1e-4,  
    warmup_steps=1500,  
    lr_scheduler_type="linear",  
    per_device_train_batch_size=batch_size,  
    per_device_eval_batch_size=batch_size,  
    weight_decay=0.01,  
    save_total_limit=10,  
    num_train_epochs=15,  
    predict_with_generate=True,  
    fp16=True,  
    save_strategy="epoch",  
    # save_steps=1000,  
    logging_dir="./logs",  
    logging_steps=100,  
    load_best_model_at_end=True,  
    metric_for_best_model="eval_loss",  
    greater_is_better=False,  
    gradient_checkpointing=False,  
    gradient_accumulation_steps=1,  
    max_grad_norm=0.75,  
    torch_compile=True  
)
```

Quá trình huấn luyện

Evaluate model: Sử dụng model có loss thấp nhất trong quá trình train, đánh giá dựa trên bộ dữ liệu Test của VLSP2023

```
def evaluate_model(
    datasets, model, tokenizer, lang_1='vi', lang_2='lo',
    multilingual=False, batch_size=32,
    separated_tokenizer=False, src_tokenizer=None
):
    device='cuda' if torch.cuda.is_available() else 'cpu'
    print('Evaluating on: ', device)

    model.to(device)
    model.eval()

    def collate_fn(batch):
        return {
            'input_ids': torch.tensor([example['input_ids'] for example in batch]),
            'attention_mask': torch.tensor([example['attention_mask'] for example in batch]),
            'labels': torch.tensor([example['labels'] for example in batch]),
        }
```

Đánh giá mô hình

Mô hình	BLEU (Sacrebleu)	BLEU (Evaluate)	CHRF (Sacrebleu)	CHRF (Evaluate)
Thang điểm	0:100	0:1	0:10	0:100
Lào-Việt	41.41	0.4188	4.97	53.29
Việt-Lào	56.55	0.2874	4.60	52.56

Kết quả đánh giá mô hình

VNU University of Engineering and Technology

**Thank
You**